

Bài 2: DANH SÁCH KẾ TIẾP

Mục tiêu:

Về kiến thức:

+ Hiểu được mô hình danh sách: Khái niệm, các dạng biểu diễn, các phép toán cơ bản

Về kỹ năng:

+ Vận dụng kiến thức đã học để cài đặt các phép toán cơ bản trên danh sách trong các bài toán cụ thể

2.1 Khái niệm danh sách

2.1.1 Khái niệm danh sách

Mô hình toán học của danh sách là một tập hợp hữu hạn biến động các phần tử thuộc cùng một lớp đối tượng nào đó (có cùng một kiểu dữ liệu).

Ta cũng lưu ý rằng một đối tượng cũng có thể xuất hiện nhiều lần trong một danh sách. Ta biểu diễn danh sách L như là một chuỗi các phần tử của nó: $a_1, a_2, \dots, a_n$ với $n \geq 0$. thì:

+ Nếu $n = 0$ ta nói *danh sách rỗng* (empty list).

+ Nếu $n > 0$ ta gọi a_1 là phần tử đầu tiên và a_n là phần tử cuối cùng của danh sách.

+ Số phần tử của danh sách ta gọi là độ dài của danh sách.

+ Một tính chất quan trọng của danh sách đó là tính tuyến tính: Các phần tử của danh sách có thứ tự tuyến tính theo vị trí (position) xuất hiện của các phần tử. Ta nói a_i đứng trước a_{i+1} , với i từ 1 đến $n-1$; Tương tự ta nói a_i là phần tử đứng sau a_{i-1} , với i từ 2 đến n . Ta cũng nói a_i là phần tử tại vị trí thứ i , hay phần tử thứ i của danh sách.

Ví dụ: Tập hợp họ tên các sinh viên của lớp TINHOC được liệt kê trên giấy như sau:

1. Nguyễn Trung Cang

2. Nguyễn Ngọc Chương

3. Lê Thị Lê Swong

4. Trịnh Vũ Thành

5. Nguyễn Phú Vĩnh

6. Phạm Quách Què

7. Vũ Xuân Trường

8. Trần Rạo Rực

là một danh sách. Danh sách này gồm có 8 phần tử, mỗi phần tử có một vị trí trong danh sách theo thứ tự xuất hiện của nó.

- *Danh sách con* : Nếu $L = (a_1, a_2, \dots, a_n)$ là một danh sách thì ta gọi là một danh sách con của L, một đoạn nào đó các phần tử kế tiếp của L. Danh sách rỗng được xem là danh sách con của một danh sách bất kỳ.

2.1.2 Các phép toán cơ bản trên danh sách

Gọi L là một danh sách đã cho, p là một vị trí (position) trong danh sách, x là một giá trị nào đó cùng kiểu với kiểu dữ liệu của các phần tử trong danh sách. Các phép toán cơ bản thường gặp trên danh sách là:

1) Chèn một phần tử vào danh sách

INSERT_LIST(x,p,L): Chèn phần tử x vào vị trí p trong danh sách L. Tức là nếu danh sách là $a_1, a_2, \dots, a_{p-1}, a_p, \dots, a_n$ thì sau khi xen ta có kết quả $a_1, a_2, \dots, a_{p-1}, x, a_p, \dots, a_n$. Nếu vị trí p không tồn tại trong danh sách thì phép toán không được xác định.

2) Tìm vị trí của một phần tử trong danh sách

LOCATE(x,L) thực hiện việc xác định vị trí phần tử có nội dung x đầu tiên trong danh sách L. Locate trả kết quả là vị trí của phần tử x trong danh sách. Nếu x không có trong danh sách thì vị trí sau phần tử cuối cùng của danh sách được trả về, tức là **ENDLIST(L)**.

3) Lấy giá trị của phần tử ở vị trí nào đó

RETRIEVE(p,L) lấy giá trị của phần tử ở vị trí p của danh sách L; nếu vị trí p không

có trong danh sách thì kết quả không xác định (có thể thông báo lỗi).

4) Xóa một phần tử ở vị trí nào đó trong danh sách

DELETE_LIST(p,L): xóa phần tử ở vị trí p trong danh sách L. Nếu vị trí p không có trong danh sách thì phép toán không được định nghĩa và danh sách L sẽ không thay đổi.

5) Tìm vị trí của phần tử đứng sau phần tử có vị trí xác định

LINK(p,L) cho kết quả là vị trí của phần tử đi sau phần tử p; nếu p là phần tử cuối cùng trong danh sách L thì LINK(p,L) cho kết quả là ENDLIST(L). Link không xác định nếu p không phải là vị trí của một phần tử trong danh sách.

6) Tìm vị trí của phần tử đứng trước phần tử có vị trí xác định **PREVIOUS(p,L)** cho kết quả là vị trí của phần tử đứng trước phần tử có vị trí p trong danh sách. Nếu p là phần tử đầu tiên trong danh sách thì Previous(p,L) không xác định. Previous cũng không xác định trong trường hợp p không phải là vị trí của phần tử nào trong danh sách.

7) Tìm vị trí của phần tử đứng đầu danh sách

FIRST(L) cho kết quả là vị trí của phần tử đầu tiên trong danh sách. Nếu danh sách rỗng thì ENDLIST(L) được trả về.

8) Kiểm tra tính rỗng của danh sách

EMPTY_LIST(L) cho kết quả TRUE nếu danh sách rỗng, ngược lại nó cho giá trị FALSE.

9) Tạo một danh sách rỗng

MAKENULL_LIST(L) khởi tạo một danh sách L rỗng, chưa có dữ liệu.

Ghi chú:

- Trong thiết kế các giải thuật sau này chúng ta dùng các phép toán trừu tượng đã được định nghĩa ở trên đây như là các phép toán nguyên thủy (cơ bản). Thật vậy, từ các phép toán nguyên thủy này ta có thể tự hình thành lên các phép toán phức tạp khác thường gặp trên danh sách như: Tạo danh sách chứa dữ liệu, sắp xếp danh sách, duyệt danh sách, tách, gộp, tính toán, tổng hợp, hiển thị danh sáchphụ thuộc vào yêu cầu cụ

thể của bài toán.

- Trên đây là các phép toán trừu tượng do chúng ta định nghĩa, nó chưa được cài đặt trong các ngôn ngữ lập trình. Do đó để thực hiện được các phép toán đó ta phải cài đặt chúng thành các chương trình con trong ngôn ngữ lập trình cụ thể. Trong bài giảng này, với mỗi cấu trúc dữ liệu cài đặt mô hình danh sách ta vẫn giữ đúng những tham số của các hàm/thủ tục đã liệt kê ở trên để thống nhất trong việc cài đặt các phép toán tương ứng với nó.

2.2 Danh sách cài đặt bằng mảng

Cài đặt danh sách bởi mảng hay còn gọi là cấu trúc dữ liệu danh sách đặc, hoặc cấu trúc dữ liệu danh sách kế tiếp, gọi tắt là: Danh sách đặc, hoặc danh sách kế tiếp, nó thuộc loại cấu trúc dữ liệu tĩnh.

2.2.1 Mô tả cài đặt

Giả sử N là số phần tử tối đa trong danh sách, với cách cài đặt này, dĩ nhiên, ta phải ước lượng số phần tử tối đa của danh sách để khai báo số phần tử của mảng cho thích hợp. Giả sử *Item* là kiểu dữ liệu của các phần tử trong danh sách. Khi đó ta sẽ biểu diễn danh sách như sau:

+ Dùng một mảng để lưu giữ các phần tử của danh (giả sử mảng *Elements*).

+ *Count* là một biến đếm số lượng phần tử hiện có trong danh sách

=> Như vậy ta có thể định nghĩa danh sách như một cấu trúc bản ghi gồm 2 trường:

Elems: Chứa các phần tử trong danh sách

size: Đếm số phần tử hiện có trong danh sách (chiều dài danh sách)

Mảng chứa các phần tử trong danh sách có dạng như sau:

Chỉ số mảng	0	1	...	size	...	N-1
Thông tin	phần tử 1	phần tử 2	...	phần tử cuối	...	...

Cài đặt

```

#define N 100 //so phan tu toi da la 100
typedef int item;
/*kieu cac phan tu la item
ma cu the o day item la kieu int */
typedef struct{
    item elems[N]; //mang kieu item
    int size; //so phan tu toi da cua mang
}List; //kieu danh sach List

```

2.2.2 Triển khai các phép toán cơ bản

1- Khởi tạo danh sách rỗng

Danh sách rỗng là một danh sách không chứa bất kỳ một phần tử nào (hay độ dài danh sách bằng 0). Theo cách khai báo trên, trường **size** chỉ vị trí của phần tử cuối cùng trong danh sách và đó cũng độ dài hiện tại của danh sách, vì vậy để khởi tạo danh sách rỗng ta chỉ việc gán giá trị trường **size** này bằng 0.

Giải thuật:

```

void init(List *L) //ham khoi tao danh sach rong
/*Danh sach L duoc khai bao kieu con tro
de khi ra khoi ham no co the thay doi duoc*/
{
    (*L).size = 0; //size = 0.
}

```

2- Kiểm tra danh sách rỗng/ đầy

Danh sách rỗng là một danh sách mà độ dài của nó bằng 0.

Danh sách đầy là một danh sách mà độ dài của nó bằng N.

Giải thuật kiểm tra tính rỗng của danh sách:

```

int isEmpty (List L){
    return (L.size==0);
}

```

Giải thuật kiểm tra tính đầy của danh sách:

```

int isFull (List L){
    return (L.size==N);
}

```

3- Nhập dữ liệu cho danh sách

Để nhập dữ liệu cho danh sách, ta nhập vào số lượng phần tử của danh sách - gán cho biến size, sau đó dùng vòng lặp nhập lần lượt từng phần tử.

Giải thuật:

```
void input (List *L){
    int n;
    printf("Nhap so phan tu cua danh sach: ");
    scanf("%d",&(*L).size);
    int i;
    for (i=0; i<(*L).size; i++){
        printf("Nhap phan tu thu %d : ",i+1);

        (*L).elems[i] = init_x();
    }
}
```

4- Thêm một phần tử vào danh sách

Trước khi chèn phần tử vào trong danh sách chúng ta nên xây dựng 1 hàm trả về dữ liệu (nhập vào dữ liệu) của phần tử cần chèn đó.

```
item init_x() //khởi tạo giá trị x{
    int temp;
    scanf("%d",&temp);
    return temp;
}
```

Sau đó tiến hành chèn:

Trong quá trình chèn chúng ta cần kiểm tra xem danh sách đầy chưa, nhập vị trí k cần chèn và kiểm tra nó, nếu phù hợp ($0 < k \leq N$) thì sẽ tiến hành chèn.

Để chèn được phần tử x vào vị trí k trong danh sách (trong mảng) ta cần dùng 1 vòng for để di chuyển các phần tử từ vị trí k về phía cuối mảng, sau đó chèn x vào vị trí k. Cuối cùng ta tăng size lên 1 đơn vị.

```

int insert_k (List *L, item x, int k){//chen x vao vi tri k
    if (isFull(*L)) //kiem tra danh sach day{
        printf("Danh sach day !");
        return 0;
    }
    if (k<1 || k>(*L).size+1){ //kiem tra dieu kien vi tri che
        printf("Vi tri chen khong hop le !\n");
        return 0;
    }
    printf ("Nhap thong tin: ");
    x = init_x(); //gan x = ham khoi tao x
    int i;
    //di chuyen cac phan tu ve cuoi danh sach
    for (i = (*L).size; i >= k; i--)
        (*L).elems[i] = (*L).elems[i-1];
    (*L).elems[k-1]=x;//chen x vao vi tri k
    (*L).size++;//tang size len 1 don vi.
    return 1;
}

```

5 - Xóa phần tử thứ k ra khỏi danh sách

Trước khi xóa ta phải kiểm tra xem danh sách có rỗng không. Nếu không rỗng ta nhập vào vị trí cần xóa và kiểm tra (phù hợp nếu $0 < k \leq N$).

Ta dùng vòng for chạy đến vị trí thứ k, sau đó dồn các phần tử từ k+1 về trước 1 đơn vị. tuy nhiên ta cần lưu lại giá trị của phần tử xóa trước khi xóa để giữ lại thông tin nếu ta cần dùng đến nó. Cuối cùng là giảm size xuống 1 đơn vị.

```

int del_k (List *L, item *x, int k){
    if (isEmpty(*L)){
        printf("Danh sach rong !");
        return 0;
    }
    if (k<1 || k>(*L).size){
        printf("Vi tri xoa khong hop le !");
        return 0;
    }
    *x=(*L).elems[k-1]; //luu lai gia tri cua phan tu can xoa
    int i;
    for (i=k-1; i<(*L).size-1; i++) //don cac phan tu ve truoc
        (*L).elems[i]=(*L).elems[i+1];
    (*L).size--; //giam size
    return 1;
}

```

6 - Tìm một phần tử trong danh sách

Ta duyệt từ đầu đến cuối danh sách nếu có giá trị x thì đưa ra vị trí của nó.

```

int search (List L, item x){
    int i;
    for (i=0; i<L.size; i++)
        if (L.elements[i] == x)
            return i+1;
    return 0;
}

```

7 - Xóa phần tử có nội dung x trong danh sách

Để xóa phần tử có nội dung x trong danh sách ta tiến hành tìm phần tử x trước bằng hàm search sau đó giá trị trả về là vị trí của x, ta tiếp tục sử dụng hàm del_k để xóa phần tử ở vị trí mà ta tìm được.

```

int del_x (List *L, item x){
    if (isEmpty(*L)){
        printf("Danh sach rong !");
        return 0;
    }
    int i = search(*L,x);
    if (!i){
        printf("Danh sach khong co %d",x);
        return 0;
    }
    do{
        del_k(L,&x,i);
        i = search(*L,x);
    }
    while (i);
    return 1;
}

```

2.3 Thực hành

2.3.1 Mục đích của bài thực hành

Bài thực hành giúp sinh viên biết cách cài đặt mô hình dữ liệu danh sách. Sau đó, sinh viên thực hành thành thạo trên các phép toán trên mô hình dữ liệu danh sách. Cuối cùng, sinh viên có khả năng vận dụng vào triển khai bài toán ứng dụng.

2.3.2 Bài thực hành mẫu

Bài 1: Cho một danh sách chứa tối đa 100 số nguyên. Yêu cầu:

- Biểu diễn danh sách bởi mảng
- Thực hiện các thao tác sau trên danh sách
- Nhập dữ liệu cho danh sách gồm n phần tử

d. ²Hiển thị danh sách vừa nhập

```
#include<stdio.h>
#include<stdlib.h>
#define N 100
typedef int item;
/*kieu cac phan tu la item
ma cu the o day item la kieu int */
typedef struct
{
    item Elems[N];
    int size;
}List;

void Init(List *L);
void Init(List *L);
int Isempty (List L);
int Isfull (List L);
void Input (List *L);
void Output (List L);
item init_x();

void Init(List *L) //ham khoi tao danh sach rong
/*Danh sach L duoc khai bao kieu con tro
de khi ra khoi ham no co the thay doi duoc*/
{
    (*L).size = 0; //size = 0.
}

int Isempty (List L)
{
    return (L.size==0);
}

int Isfull (List L)
{
    return (L.size==N);
}

item init_x() //khoi tao gia tri x
{
    int temp;
    scanf("%d",&temp);
```

```

    return temp;
}

void Input (List *L)
{
    int n;
    printf("Nhap so phan tu cua danh sach: ");
    scanf("%d",&(*L).size);
    int i;
    for (i=0; i<(*L).size; i++)
    {
        printf("Nhap phan tu thu %d : ",i+1);
        (*L).Elems[i] = init_x();
    }
}

void Output (List L)
{
    printf("Danh sach: \n");
    int i;
    for (i=0; i<L.size; i++)
        printf("%5d",L.Elems[i]);
    printf("\n");
}

int main()
{
    List L;
    Init(&L);
    Input(&L);
    Output(L);
}

```

2.3.3 Bài tập luyện tập

Bài 2: Cho một danh sách chứa tối đa m số nguyên. Yêu cầu:

1. Biểu diễn danh sách bởi mảng
2. Thực hiện các thao tác sau trên danh sách:
 - a. Thêm phần tử có giá trị x vào vị trí thứ k trong danh sách
 - b. Xóa phần tử ở vị trí thứ k trong danh sách
 - c. Tìm kiếm phần tử có giá trị x trong danh sách

Bài 3: Cho một danh sách chứa tối đa m số nguyên. Yêu cầu:

1. Biểu diễn danh sách bởi mảng
2. Thực hiện các thao tác sau trên danh sách:
 - a. Đếm số phần tử: chẵn, lẻ, âm, dương có trong danh sách
 - b. Tính TBC các số nguyên dương trong danh sách
 - c. Tách danh sách ban đầu thành các danh sách chứa các số: chẵn, lẻ, âm, dương.

Bài 4: Giả sử cần quản lý k phòng ban, và danh sách các cán bộ trong từng phòng ban của một cơ quan. Anh (chị hãy):

1. Biểu diễn danh sách bằng mảng
2. Thực hiện các thao tác:
 - a. Nhập dữ liệu cho từng danh sách
 - b. Hiển thị thông tin các cán bộ công tác tại 2 phòng ban trở nên
 - c. Liệt kê các lãnh đạo của từng phòng ban
 - d. Hiển thị các cán bộ có số năm công tác từ 10 năm trở nên

Bài 5: Cho một danh sách chứa tối đa m sinh viên. Mỗi sinh viên cần quản lý các thông tin Họ tên, giới tính, lớp, điểm tổng kết. Yêu cầu:

1. Biểu diễn danh sách bởi mảng
2. Thực hiện các thao tác sau trên danh sách
 - a. Nhập dữ liệu cho danh sách gồm n phần tử
 - b. Hiển thị danh sách vừa nhập.
 - c. Thêm một sinh viên vào cuối danh sách
 - d. Đếm số lượng sinh viên nữ có điểm tổng kết > 8.0
 - e. Xóa sinh viên có họ tên là x do bỏ học lâu ngày
 - f. Tách danh sách sinh viên theo từng lớp